

BomberArena

プログラム説明資料

ジャンル: マルチプレイ アクション/パーティーゲーム

開発環境: C# / Unity | 開発期間: 2025/12~2026/2

ゲーム科 3年 石川原 吉美

C#

Unity

Input System

Tilemap



アピールポイント

01 MatchManager 設計

ラウンド進行・生存判定・勝利数管理を1クラスに集約。静的プロパティ InputLocked で全プレイヤーの入力を一括制御する。

02 MovementController

ボンバーマン式斜め移動禁止を Rigidbody2D で実装。CPU 対応の SetMoveInput() で入力ルートを一統し、アニメも確実に更新。

03 BombController

爆弾設置・4方向爆風伝播・Tilemap タイル破壊を再帰処理で実装。RuntimeRoot 管理でラウンド後の残留オブジェクトをゼロにする。

04 StageResetter

ラウンド毎に Tilemap を完全復元。RuntimeRoot/ItemsRoot で生成物を一括管理し、ResetStage() 1 回で初期状態に戻す。

05 GameSettings (static)

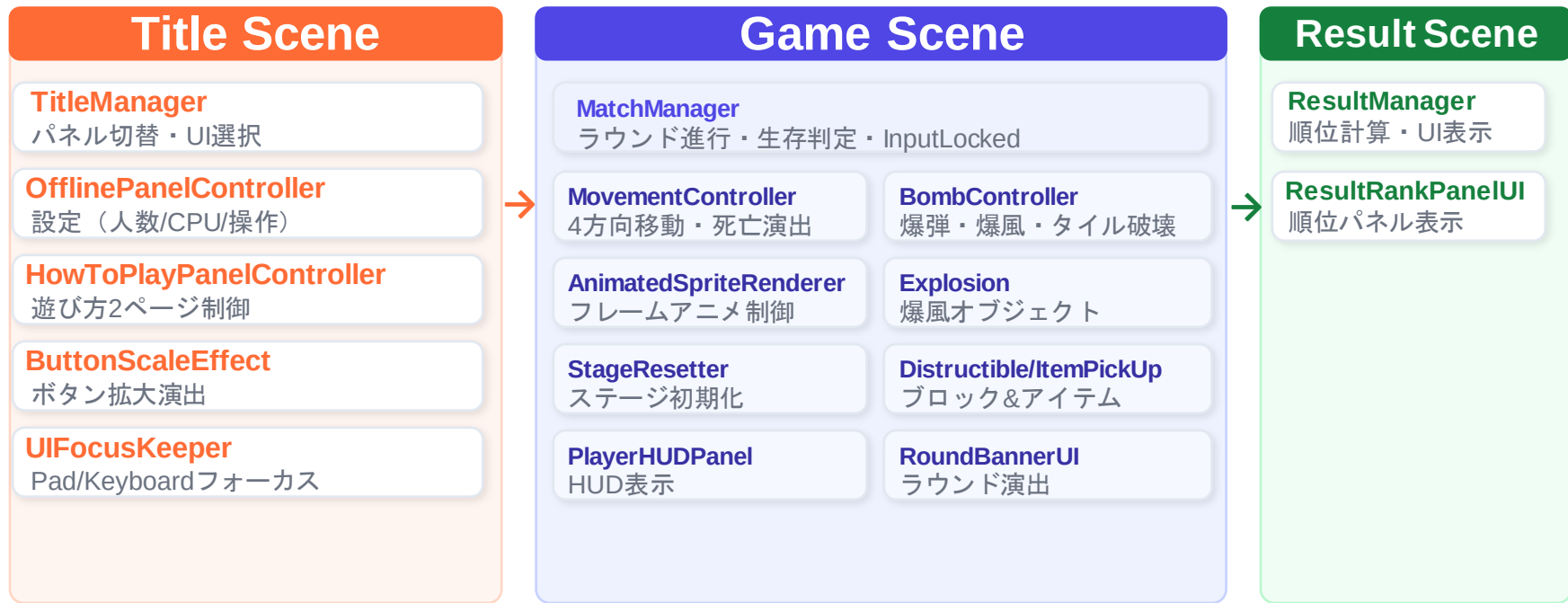
人数・CPU/Human・操作方法を static クラスでシーン間に永続保持。MatchResultData も同様に試合結果を受け渡す。

06 UIシステム

タイトル・設定・HUD・リザルトを独立管理。UIFocusKeeper で Gamepad/Keyboard 両対応のフォーカスを毎フレーム自動復元する。

ゲームシステム — アーキテクチャ全体図

アーキテクチャ



static GameSettings 人数・CPU・操作設定 (全シーンから参照) | **MatchResultData** 試合結果 (GameScene → ResultScene)

MatchManager 設計

ゲーム進行管理 — ラウンド制御・生存判定・InputLocked

ラウンド制御

currentRound を加算し StartRoundRoutine コルーチンで演出 → ゲーム開始を順番に処理。

生存判定

Update() で activeSelf なプレイヤー数を毎フレーム計測。1人以下になったら OnRoundWin() を呼ぶ。

勝利数管理

wins[i] を加算し HUD に反映。3勝に達したら MatchResultData に保存してリザルトへ遷移。

InputLocked

static プロパティで全スクリプトから参照可能。演出中は MovementController・BombController が入力を無視する。

// MatchManager.cs 抜粋

```
private IEnumerator StartRoundRoutine()
{
    currentRound++;
    roundEnded = false;
    stageResetter.ResetStage(); // マップ初期化
    ResetPlayersForNewRound(); // 全員復活
    InputLocked = true;
    Time.timeScale = 0f;
    yield return StartCoroutine(
        roundBannerUI.ShowRoundThenGo(currentRound));
    Time.timeScale = 1f;
    InputLocked = false;
}
```

// Update() 生存判定

```
int aliveCount=0, lastAlive=-1;
for(int i=0; i<players.Length; i++)
    if(players[i]!=null && players[i].activeSelf)
        { aliveCount++; lastAlive=i; }

if(aliveCount<=1 && lastAlive!=-1)
{
    roundEnded = true;
    InputLocked = true;
    Time.timeScale = 0f;
    OnRoundWin(lastAlive);
}
```

MatchManager — 試合フロー図

TitleScene → GameScene (Round ループ) → ResultScene

Title Scene



Game Scene (Round 1 ~ N)



Result Scene

設定フロー

OfflinePanelController

人数 / CPU / 操作方法
を設定・確定



GameSettings (static)

設定値を静的クラスに
書き込み保持



SceneManager .LoadScene()

"GameScene" へ
遷移してゲーム開始

3勝先取 / 勝者が決まるまでラウンドを繰り返す

InputLocked = true

ラウンド演出中・操作不可



Round X → GO !

WaitForSecondsRealtime / timeScale=0 でも動く



InputLocked = false / ゲーム進行

プレイヤー操作可・爆弾設置・アイテム取得



生存 1 人以下 → onRoundWin()

勝者の wins++ / MatchManager が勝敗判定

🔄 wins < 3 : 次ラウンドへ

wins ≥ 3 : ResultScene →

集計フロー

MatchResultData

wins[] を static
クラスから読み込み



順位計算

wins 降順ソートで
1位~4位を決定



ResultRankPanelUI .Set()

- ・ 順位・アイコン
- ・ 勝利数を UI に表示

WaitForSecondsRealtime を使用 → Time.timeScale=0 中でも演出タイマーが進む (TimeScale依存の WaitForSeconds は停止するため)

MovementController — プレイヤー移動

操作・アニメーション・死亡演出

斜め禁止

X/Yの大きい軸のみ採用。SetDirectionInternal()で direction と表示スプライトを同時に更新。

Rigidbody2D

FixedUpdate で MovePosition を使用。物理エンジンとの衝突判定を維持しつつ安定した移動を実現。

CPU対応

SetMoveInput(Vector2) でInputSystemを介さずに入力を注入。OnMove()も同関数に委譲して処理統一。

死亡シーケンス

爆風タグ検出 → 移動/爆弾無効化 → 死亡アニメ
→ 1.25秒後に SetActive(false) でラウンド終了通知。

// MovementController.cs 抜粋

```
public void SetMoveInput(Vector2 input)
{
    if (MatchManager.InputLocked) return;
    if (isDead) return;
    if (input.sqrMagnitude < 0.01f)
    {
        direction = Vector2.zero;
        if (activeSpriteRenderer != null)
            activeSpriteRenderer.idle = true;
        return;
    }
    // 斜め禁止：大きい軸だけ採用
    if (Mathf.Abs(input.x) > Mathf.Abs(input.y))
        SetDirectionInternal(
            input.x > 0 ? Vector2.right : Vector2.left,
            input.x > 0 ? spriteRendererRight :
spriteRendererLeft);
    else
        SetDirectionInternal(
            input.y > 0 ? Vector2.up : Vector2.down,
            input.y > 0 ? spriteRendererUp :
spriteRendererDown);
}
```

BombController — 爆弾システム

設置・爆風伝播・タイル破壊・アイテム拡張

グリッド吸着

PlaceBomb()でプレイヤー座標を Mathf.Round してマス目に揃えてから Instantiate する。

4方向爆風

Explode()を再帰呼び出し。物理OverlapBoxで壁・ブロックを検出し、当たったら伝播を止める。

タイル破壊

ClearDestructible()でWorldToCell変換
→GetTile→SetTile(null)。破壊演出はRuntimeRoot配下に生成。

RuntimeRoot管理

爆弾・爆風・演出を RuntimeRoot 配下に生成。ラウンドリセット時に一括 Destroy できる設計。

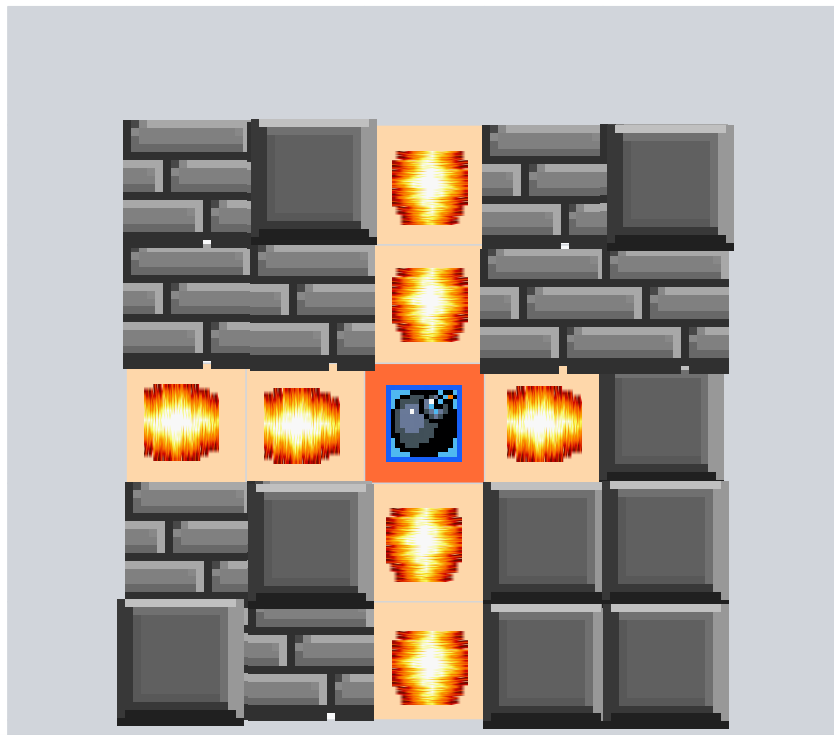
// BombController.cs 抜粋

```
private void Explode(Vector2 pos,
                    Vector2 dir, int len)
{
    if(len <= 0) return;
    pos += dir;
    // 壁/ブロックに当たったら終了
    if(Physics2D.OverlapBox(pos,
        Vector2.one/2f, 0f, layerMask))
    {
        ClearDestructible(pos); // タイル破壊
        return;
    }
    Explosion ex = Instantiate(explosionPrefab,
                               pos,
                               Quaternion.identity);

    ex.transform.SetParent(stageResetter.RuntimeRoot, true);
    ex.SetActiveRenderer(len>1? ex.middle : ex.end);
    ex.SetDirection(dir);
    ex.DestroyAfter(explosionDuration);
    Explode(pos, dir, len-1); // 再帰
}
```

BombController — 爆風伝播概念図

土管ライフサイクル



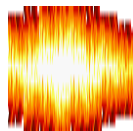
凡例



壁（爆風を遮断）



爆弾（起点）



爆風（伝播マス）



破壊ブロック
（伝播を止め破壊）

ポイント: Explode() を再帰呼び出し。Physics2D.OverlapBox で壁/ブロックを検出 → ブロックなら ClearDestructible() でタイルを削除して終了。壁なら何もせず終了。それ以外は爆風を生成して次のマスへ再帰。

StageResetter & Distructible — ステージ管理

破壊ブロック復元・生成物一括削除・アイテムドロップ

StageResetter.cs

① タイル復元

Awake() で GetTilesBlock() を使い初期タイルを savedTiles に保存。ResetStage() で ClearAllTiles → SetTilesBlock → Refresh で完全復元。

② アイテム削除

ItemsRoot の子オブジェクトをすべて Destroy。ラウンド中に落ちたアイテムを確実に除去。

③ 爆弾・爆風削除

RuntimeRoot の子をすべて Destroy。ラウンド終了直後に残留する爆弾・爆風・演出を一括消去。

Distructible.cs

アイテムドロップ

Start() で Destroy(gameObject, destructionTime) を登録。OnDestroy() で itemSpawnChance(0-1) の確率でランダムアイテムを生成し ItemsRoot 配下に入れる。

// StageResetter.cs 抜粋

```
public void ResetStage()
{
    // 1) タイル復元
    destructiblesTilemap.ClearAllTiles();
    destructiblesTilemap.SetTilesBlock(
        savedBounds, savedTiles);
    destructiblesTilemap.RefreshAllTiles();

    // 2) アイテム削除
    ClearChildren(itemsRoot);

    // 3) 爆弾・爆風削除
    ClearChildren(runtimeRoot);
}

private void ClearChildren(Transform root)
{
    for(int i=root.childCount-1;i>=0;i--)
        Destroy(root.GetChild(i).gameObject);
}
```

GameSettings & MatchResultData — データ管理

staticクラスによるシーン間データ受け渡し

GameSettings.cs

playerCount

参加人数（2～4）。OfflinePanelControllerで変更。

SlotType[]

Human/CPU判定。MatchManagerで
MovementController/BombControllerの有効化を切り替える。

ControlType[]

Keyboard/Gamepad。MatchManagerの
ConfigurePlayerInputsでデバイスを割り当てる。

ResetDefaults()

タイトルに戻る際に全設定を初期値に戻す。ResultManager
から呼び出す。

MatchResultData.cs

シーン間データ受け渡し

GameScene終了時にSetFrom()で人数・勝利数を保存。
ResultScene開始時に参照して順位を計算。staticなのでシーンをまたいでもデータが保持される。

// GameSettings.cs 抜粋

```
public static class GameSettings
{
    public enum SlotType { Human, CPU }
    public enum ControlType { Keyboard, Gamepad }
    public enum CpuDifficulty { Easy, Normal, Hard }

    public static int playerCount = 2;
    public static SlotType[] slotTypes =
        new SlotType[4]; // 全員 Human
    public static ControlType[] controlTypes =
        new ControlType[4]; // P1=Keyboard 他=Gamepad

    public static void ResetDefaults()
    {
        playerCount = 2;
        // 全員 Human + 初期操作方法に戻す
    }
}
```

UIシステム — 画面管理

タイトル・設定・HUD・リザルトの独立管理

タイトル画面

TitleManager

Title / Offline / HowToPlay の3パネルを ShowOnly() で排他切り替え。SetSelectedNextFrame()でSetActive直後のフォーカス外れを防ぐ。

HUD

PlayerHUDPanel

Bind(player)でBombController/MovementControllerを取得。Update()で毎フレームアイテム数を RefreshItemCounts()で更新。SetWins()はMatchManagerから呼ぶ。

ラウンド演出

RoundBannerUI

ShowRoundThenGo()コルーチン。Time.timeScale=0中でも WaitForSecondsRealtimeで正確に時間を計測。CanvasGroupのalphaでフェードアウト。

オフライン設定

OfflinePanelController

人数・CPU/Human・操作方法を GameSettings に書き込み RefreshImages()でUIを即時反映。キーボード同時使用制限をUI側でDeny/Swapモードで制御。

リザルト

ResultManager

MatchResultDataのwins[]をSort(降順→番号昇順)して順位を決定。ResultRankPanelUI.Set()で順位・アイコン・勝利数を表示する。

フォーカス維持

UIFocusKeeper

Update()でGamepad/Keyboardの入力を検出し、currentSelectedGameObjectがnullなら fallbackSelected を強制選択。GamepadとKeyboardの両方に対応。

まとめ

● MatchManager

ラウンド進行・生存判定・InputLockedによる一元管理で、試合の状態遷移を安全に制御

● MovementController

Rigidbody2Dとボンバーマン式斜め禁止で直感的な操作を実現。CPU入力口も統一

● BombController

再帰処理による4方向爆風とRuntimeRoot管理でラウンド間の残留をゼロに

● StageResetter

TilemapのスナップショットとRoot一括削除で毎ラウンド確実に初期化

● static設計

GameSettings/MatchResultDataのstaticクラスがシーン間のデータ受け渡しを単純化